P03 Advanced Movement

CRE132 worksheet

What you'll make

Two new moves for your platformer player: a **double jump** and a **dash**. The double jump goes into a new controller, `AdvancedController`, made from a copy of P02's `BetterJumpController`. The dash is a separate script that takes over from the controller for a moment.

By the end you'll know how an `if` / `else if` chain decides which jump happens, and how one script can switch another off with `enabled = false`.

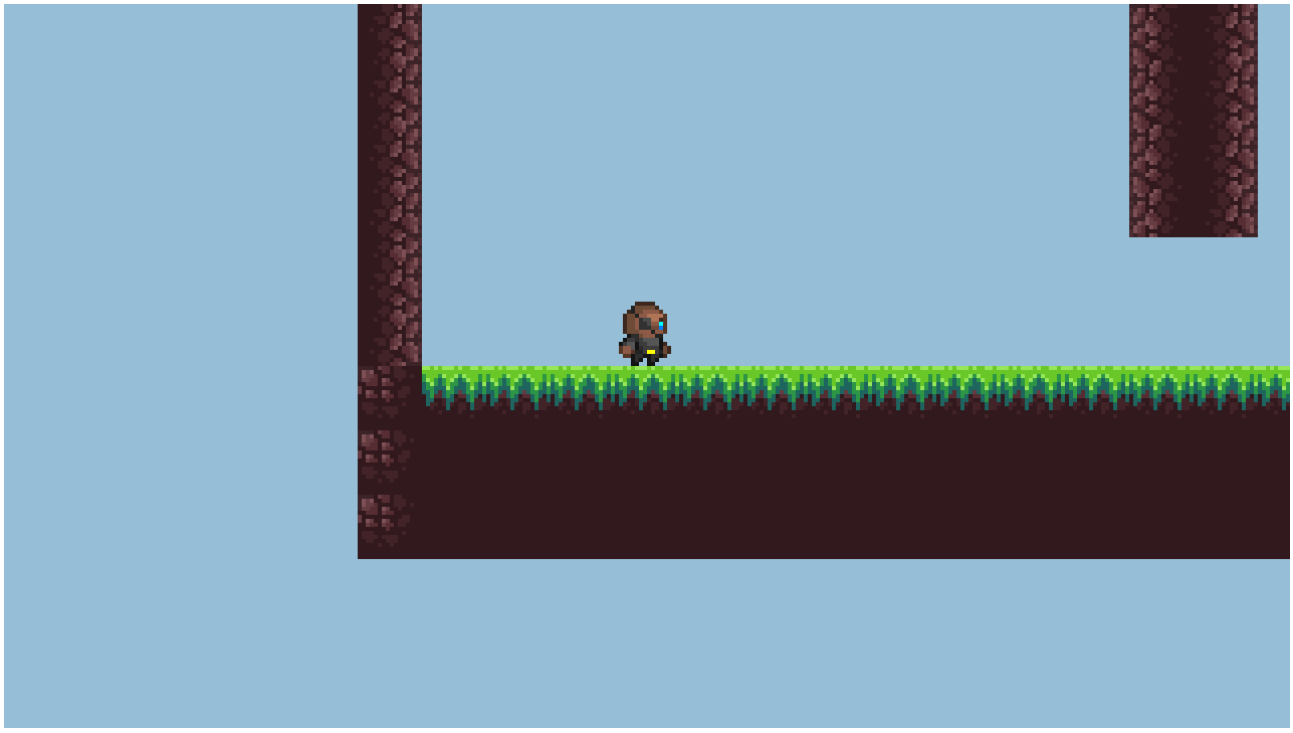


Figure 1: The start of the level: a tall shaft on the left, with a high step and a long gap further along.

Before you start

1. Unzip `CRE132_P03_AdvancedMovement_Starter.zip`. In Unity Hub, choose **Add > Add project from disk** and pick the `CRE132_P03_AdvancedMovement_Starter` folder. Open it with Unity **6000.3.11f1**.
2. Open the scene `Assets/_game/Scenes/Platformer/P03_AdvancedMovement`.
3. Press **Play**. The player stands on the ground but can't move: it has no controller yet. Press Play again to stop.

The shaft on the left is too high to climb for now: P03b's wall jump will climb it. You'll also need your `BetterJumpController.cs` from P02. If yours doesn't work, use the one in `CRE132_P02_BetterJump_Finished` (in `Assets/_game/Scripts/P02_BetterJump`).

> **Making a script:** in the Project window, go to the folder, right-click and choose **Create > Scripting > MonoBehaviour Script**. Type the name straight away and press Enter. The name must match the class name in the code exactly, capital letters included.

1. Start from a copy: AdvancedController

Make a folder `Assets/_game/Scripts/P03_AdvancedMovement` and create a script in it called `AdvancedController`. Double-click it to open it.

In your code editor, also open your P02 `BetterJumpController.cs` (**File > Open File**). Select all of it, copy it, and paste it over everything in `AdvancedController`. Change the class line to `public class AdvancedController : MonoBehaviour`, and change the comment at the top to say what this script is.

Add it: select the **Player** and **Add Component > Advanced Controller**. Fill the slots as in P02: drag the **Ground Check** child into **Ground Check**, set **Ground Layers** to **Ground**, and pick **Player/Move** and **Player/Jump** for **Move Action** and **Jump Action**.

Test it: press Play. Everything from P02 works: variable jump, faster fall, coyote time and jump buffering. Now add the new moves one at a time.

2. Double jump

The high step can't be reached with one jump. Add a new group of fields below `jumpBufferTime`:

```
    [Header("Double Jump")]
    [SerializeField] private int extraJumps = 1;
```

Add a counter below `private float jumpBufferCounter = -1f;`:

```
    private int extraJumpsLeft;
```

In `FixedUpdate`, in the coyote time code, refill the extra jumps while on the ground. Add the middle line here:

```
        if (isGrounded)
        {
            coyoteCounter = coyoteTime;
            extraJumpsLeft = extraJumps;
        }
```

Now replace P02's jump. Delete the whole jump `if` (from `if (jumpBufferCounter >= 0f && coyoteCounter >= 0f)` to its closing brace) and put this in its place:

```
        bool wantsToJump = jumpBufferCounter >= 0f;
        if (wantsToJump && coyoteCounter >= 0f)
        {
            // A normal jump from the ground (or just after leaving it).
            velocity.y = jumpSpeed;
            jumpBufferCounter = -1f;
            coyoteCounter = -1f;
        }
```

Then, straight after that closing brace, add the double jump:

```
        else if (wantsToJump && extraJumpsLeft > 0)
        {
            // Double jump: a jump in the air uses up one extra jump until we land again.
            velocity.y = jumpSpeed;
            extraJumpsLeft = extraJumpsLeft - 1;
            jumpBufferCounter = -1f;
        }
```

- **`wantsToJump`** is a bool with a readable name for "Jump was pressed recently". Both jumps check it.
- **`extraJumpsLeft`** goes down by one for each jump in the air and is filled back up to `extraJumps` when you land.
- **Order matters:** the ground jump (with coyote time) is checked first. The double jump is an `else if`, so it's only tried when the ground jump didn't happen. That way a jump just after running off a ledge is a normal jump and doesn't use up the extra one.

Test it: jump, then press Jump again in the air. Climb the high step. Set **Extra Jumps** to **2** for a triple jump, and **0** to switch it off. Set it back to **1**.

3. Which way is the player facing?

The dash needs to know which way to go. Add this below `private int extraJumpsLeft;`:

```
    // 1 = facing right, -1 = facing left.
    private float facingDirection = 1f;
```

Add a property below the `IsGrounded` property, so the Dash script can read it:

```
    // The Dash script reads this to dash the way the player faces.
    public float FacingDirection
    {
        get { return facingDirection; }
    }
```

In `Update`, straight below the line `horizontalInput = moveAction.action.ReadValue<Vector2>().x;`, add:

```
        if (horizontalInput > 0.1f)
        {
            facingDirection = 1f;
        }
        else if (horizontalInput < -0.1f)
        {
            facingDirection = -1f;
        }
```

- **`facingDirection`** remembers the last way you pushed: 1 for right, −1 for left. Input below 0.1 either way counts as none, so letting go keeps the old direction.
- **`FacingDirection`** has only a `get`, like `IsGrounded`: other scripts can read it but not change it. P03b's wall check will use it too.

Test it: press Play and check the Console has no errors. There's nothing new to see yet: the dash uses this next.

4. The Dash script

The dash is a separate script, so the controller doesn't get even longer. Create a script `Dash` in the `P03_AdvancedMovement` folder:

```
// A quick dash the way the player faces. While dashing, gravity is off and the
// controller script is switched off, so nothing else changes the velocity. Then
// everything is switched back on.
using System.Collections;
using UnityEngine;
using UnityEngine.InputSystem;

public class Dash : MonoBehaviour
```

```
{
    [SerializeField] private float dashSpeed = 20f;
    [SerializeField] private float dashTime = 0.15f;
    [SerializeField] private float cooldown = 0.5f;
    [SerializeField] private InputActionReference dashAction;

    private Rigidbody2D body;
    private AdvancedController controller;
    private bool canDash = true;

    private void Awake()
    {
        body = GetComponent<Rigidbody2D>();
        controller = GetComponent<AdvancedController>();
    }

    private void OnEnable()
    {
        dashAction.action.Enable();
    }

    private void OnDisable()
    {
        dashAction.action.Disable();
    }

    private void Update()
    {
        if (dashAction.action.WasPressedThisFrame() && canDash)
        {
            StartCoroutine(DashRoutine());
        }
    }

    private IEnumerator DashRoutine()
    {
        canDash = false;
        float startGravity = body.gravityScale;

        controller.enabled = false;
        body.gravityScale = 0f;
        body.linearVelocity = new Vector2(controller.FacingDirection * dashSpeed, 0f);

        yield return new WaitForSeconds(dashTime);

        body.gravityScale = startGravity;
        body.linearVelocity = new Vector2(0f, 0f);
        controller.enabled = true;

        yield return new WaitForSeconds(cooldown);
        canDash = true;
    }
}
```

- **`controller.enabled = false`** switches the AdvancedController off, which stops its `Update` and `FixedUpdate`. Nothing else changes the velocity while the dash runs. It's a simple way for one ability to take over for a moment.
- **Gravity 0** keeps the dash flat. The old gravity is saved in `startGravity` and put back afterwards.

- **The coroutine** (`IEnumerator` with `yield return`) waits `dashTime` without stopping the game, then puts everything back, then waits `cooldown` before you may dash again. `canDash` stops dash spamming.

First, a Dash action. Double-click `CRE132Controls` (in `Assets/_game/Input`) and add an action called `Dash` (**Button**) with three bindings: **K [Keyboard]**, **Left Control [Keyboard]** and **Right Shoulder [Gamepad]**. Set the first binding's **Path** with **Listen**, then use **+ > Add Binding** for the other two. Click **Save Asset** at the top of the window (or tick **Auto-Save**) and close the editor.

Add it to the **Player**. Leave **Dash Speed** at **20**, **Dash Time** at **0.15** and **Cooldown** at **0.5**. In the **Dash Action** slot, click the small circle and pick **Player/Dash** (from `CRE132Controls`).

Test it: press **K** to dash the way you face. Cross the long gap with a jump, a dash and a double jump. If you fall in, steps lead back up.

Check your work

- ☐ `CRE132Controls` has a **Dash** action bound to K, Left Control and Right Shoulder.
- ☐ You can jump once more in the air, and the high step can be reached.
- ☐ Running off a ledge and jumping late still leaves you a double jump.
- ☐ K dashes sideways in a straight line, the way you face, and you can't dash again straight away.
- ☐ Everything from P02 still works.
- ☐ Compare with `CRE132_P03_AdvancedMovement_Finished` if something behaves differently.

Key points

- **Switching a script off with `enabled = false`** stops its `Update` and `FixedUpdate`. One ability can take over for a moment and then hand back.
- **A cooldown timer** makes a move feel deliberate instead of spammy.
- **Each move is only a few lines; the hard part is how they work together.** The order of the `if` / `else if` chain decides which jump wins. Test the moves together, not just one at a time.

If something goes wrong

- **The script won't go on the Player:** after pasting P02's code, the class line still says `BetterJumpController`. Change it to `AdvancedController`.
- **"'AdvancedController' does not contain a definition for 'FacingDirection'" in Dash:** the `FacingDirection` property is missing from AdvancedController.
- **NullReferenceException in Dash:** the Dash Action slot is empty, or the Player has no Advanced-Controller.
- **"'else' without 'if'" or similar errors:** the double jump `else if` has been pasted somewhere other than straight after the closing brace of the normal jump.
- **The dash always goes right:** the facing code is missing from `Update`.

Try it

1. Allow only one dash in the air: reset `canDash` only when the controller's `IsGrounded` is true.
2. Add a dash trail: briefly spawn faded copies of the sprite while dashing (E05 or S07's pool can help).